

Nama : Deswita Khansa Rafifah

NIM : 254107020151

Kelas : TI-G

LAPORAN JOBSHEET 14

14.2.1 Percobaan 1

➤ Kode Program

> Kode Program class Mahasiswa06.java

```
package Pertemuan14;
```

```
public class Mahasiswa06 {  
    String nim;  
    String nama;  
    String kelas;  
    double ipk;
```

```
    public Mahasiswa06() {  
  
    }
```

```
    public Mahasiswa06(String nim, String nama, String kelas, double ipk) {  
        this.nim = nim;  
        this.nama = nama;  
        this.kelas = kelas;  
        this.ipk = ipk;  
    }
```

```
    public void tampilInformasi() {  
        System.out.println(  
            "NIM: " + this.nim + " | "+  
            "Nama: " + this.nama + " | "+  
            "Kelas: " + this.kelas + " | "+  
            "IPK: " + this.ipk);  
    }  
}
```

> Kode Program class Node06.java

```
package Pertemuan14;
```

```
public class Node06 {  
    Mahasiswa06 mahasiswa;
```

```

Node06 left, right;

public Node06() {

}

public Node06(Mahasiswa06 mahasiswa) {
    this.mahasiswa = mahasiswa;
    left = right = null;
}
}

```

> Kode Program class BinaryTree06.java

```

package Pertemuan14;

public class BinaryTree06 {
    Node06 root;

    public BinaryTree06() {
        root = null;
    }

    public boolean isEmpty() {
        return root == null;
    }

    public void add(Mahasiswa06 mahasiswa) {
        Node06 newNode = new Node06(mahasiswa);
        if (isEmpty()) {
            root = newNode;
        } else {
            Node06 current = root;
            Node06 parent = null;
            while(true) {
                parent = current;
                if (mahasiswa.ipk < current.mahasiswa.ipk) {
                    current = current.left;
                    if (current == null) {
                        parent.left = newNode;
                        return;
                    }
                } else {
                    current = current.right;
                    if (current == null) {

```

```

        parent.right = newNode;
        return;
    }
}
}
}
}

boolean find (double ipk) {
    boolean result = false;
    Node06 current = root;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            result = true;
            break;
        } else if (ipk > current.mahasiswa.ipk) {
            current = current.right;
        } else {
            current = current.left;
        }
    }
    return result;
}

void traversePreOrder (Node06 node) {
    if (node != null) {
        node.mahasiswa.tampilInformasi();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

void traverseInOrder (Node06 node) {
    if (node != null) {
        traverseInOrder(node.left);
        node.mahasiswa.tampilInformasi();
        traverseInOrder(node.right);
    }
}

void traversePostOrder (Node06 node) {
    if (node != null) {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
    }
}

```

```

        node.mahasiswa.tampilInformasi();
    }
}

```

```

Node06 getSuccessor (Node06 del) {
    Node06 successor = del.right;
    Node06 successorParent = del;
    while (successor.left != null) {
        successorParent = successor;
        successor = successor.left;
    }
    if (successor != del.right) {
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}

```

```

void delete(double ipk) {
    if (isEmpty()) {
        System.out.println("Binary tree kosong");
        return;
    }
    Node06 parent = root;
    Node06 current = root;
    boolean isLeftChild = false;
    while (current != null) {
        if (current.mahasiswa.ipk == ipk) {
            break;
        } else if (ipk < current.mahasiswa.ipk) {
            parent = current;
            current = current.left;
            isLeftChild = true;
        } else if (ipk > current.mahasiswa.ipk) {
            parent = current;
            current = current.right;
            isLeftChild = false;
        }
    }
    if (current == null) {
        System.out.println("Data tidak ditemukan");
        return;
    } else {
        if (current.left == null && current.right == null) {

```

```

    if (current == root) {
        root = null;
    } else {
        if (isLeftChild) {
            parent.left = null;
        } else {
            parent.right = null;
        }
    }
} else if (current.left == null) {
    if (current == root) {
        root = current.right;
    } else {
        if (isLeftChild) {
            parent.left = current.right;
        } else {
            parent.right = current.right;
        }
    }
} else if (current.right == null) {
    if (current == root) {
        root = current.left;
    } else {
        if (isLeftChild) {
            parent.left = current.left;
        } else {
            parent.right = current.left;
        }
    }
} else {
    Node06 successor = getSuccessor(current);
    System.out.println("Jika 2 anak, current = ");
    successor.mahasiswa.tampilInformasi();
    if (current == root) {
        root = successor;
    } else {
        if (isLeftChild) {
            parent.left = successor;
        } else {
            parent.right = successor;
        }
    }
    successor.left = current.left;
}

```

```

    }
}
}

```

> Kode Program class BinaryTreeMain06.java

```

package Pertemuan14;

public class BinaryTreeMain06 {
    public static void main(String[] args) {

        BinaryTree06 bst = new BinaryTree06();

        bst.add(new Mahasiswa06("244160121", "Ali", "A", 3.57));
        bst.add(new Mahasiswa06("244160221", "Badar", "B", 3.85));
        bst.add(new Mahasiswa06("244160185", "Candra", "C", 3.21));
        bst.add(new Mahasiswa06("244160220", "Dewi", "B", 3.54));

        System.out.println("\nDaftar semua mahasiswa (in order traversal):");
        bst.traverseInOrder(bst.root);

        System.out.println("\nPencarian data mahasiswa:");
        System.out.print("Cari mahasiswa dengan ipk: 3.54 : ");
        String hasilCari = bst.find(3.54)?"Ditemukan":"Tidak ditemukan";
        System.out.println(hasilCari);

        System.out.print("Cari mahasiswa dengan ipk: 3.22 : ");
        hasilCari = bst.find(3.22)?"Ditemukan":"Tidak ditemukan";
        System.out.println(hasilCari);

        bst.add(new Mahasiswa06("244160131", "Devi", "A", 3.72));
        bst.add(new Mahasiswa06("244160205", "Ehsan", "D", 3.37));
        bst.add(new Mahasiswa06("244160170", "Fizi", "B", 3.46));

        System.out.println("\nDaftar semua mahasiswa setelah penambahan 3
        mahasiswa:");
        System.out.println("InOrder Traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println("\nPreOrder Traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println("\nPostOrder Traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println("\nPenghapusan data mahasiswa");
    }
}

```

```

        bst.delete(3.57);
        System.out.println("\nDaftar semua mahasiswa setelah penghapusan 1 mahasiswa
(in order traversal):");
        bst.traverseInOrder(bst.root);
    }
}

```

➤ Hasil Running

Daftar semua mahasiswa (in order traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
 NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pencarian data mahasiswa:

Cari mahasiswa dengan ipk: 3.54 : Ditemukan
 Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:

InOrder Traversal:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
 NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
 NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
 NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

PreOrder Traversal:

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
 NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
 NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
 NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85
 NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

PostOrder Traversal:

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46
 NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37
 NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54
 NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85
NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

Penghapusan data mahasiswa

Jika 2 anak, current =

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pertanyaan

1. Mengapa dalam binary search tree proses pencarian data bisa lebih efektif dilakukan dibanding binary tree biasa?

Jawab: Karena pada Binary Search Tree (BST), data disusun secara terurut. Nilai yang lebih kecil dari root disimpan di subtree kiri, sedangkan nilai yang lebih besar disimpan di subtree kanan. Dengan aturan tersebut, proses pencarian tidak perlu memeriksa semua node satu per satu. Program cukup membandingkan nilai yang dicari dengan node saat ini, lalu bergerak ke kiri atau kanan sesuai kondisi. Hal ini membuat proses pencarian menjadi lebih cepat dan efisien dibanding binary tree biasa yang tidak memiliki aturan pengurutan data.

2. Untuk apakah di class Node, kegunaan dari atribut left dan right?

Jawab: Atribut left digunakan untuk menyimpan referensi atau alamat node anak sebelah kiri, sedangkan atribut right digunakan untuk menyimpan referensi node anak sebelah kanan. Kedua atribut tersebut berfungsi untuk menghubungkan antar node sehingga terbentuk struktur binary tree.

3. a. Untuk apakah kegunaan dari atribut root di dalam class BinaryTree?

Jawab: Atribut root digunakan untuk menyimpan node paling atas atau node utama dalam binary tree. Semua proses dalam tree seperti penambahan data, pencarian data, traversal, dan penghapusan node selalu dimulai dari root.

- b. Ketika objek tree pertama kali dibuat, apakah nilai dari root?

Jawab: Ketika objek tree pertama kali dibuat, nilai root adalah null karena tree masih kosong dan belum memiliki node.

4. Ketika tree masih kosong, dan akan ditambahkan sebuah node baru, proses apa yang akan terjadi?

Jawab: Ketika tree masih kosong dan ditambahkan node baru, maka node tersebut akan langsung dijadikan sebagai root. Hal ini terjadi karena belum ada node lain di dalam tree sehingga node pertama otomatis menjadi akar dari binary tree.

5. Perhatikan method add(), di dalamnya terdapat baris program seperti di bawah ini. Jelaskan secara detil untuk apa baris program tersebut?

```
parent = current;
if (mahasiswa.ipk < current.mahasiswa.ipk) {
    current = current.left;
    if (current == null) {
        parent.left = newNode;
        return;
    }
} else {
    current = current.right;
    if (current == null) {
        parent.right = newNode;
        return;
    }
}
```

Jawab: Baris program tersebut digunakan untuk menentukan posisi node baru di dalam Binary Search Tree berdasarkan nilai IPK mahasiswa. Jika IPK data baru lebih kecil dari IPK node saat ini (current), maka proses berpindah ke child kiri (current.left). Sebaliknya, jika IPK lebih besar atau sama, maka proses berpindah ke child kanan (current.right). Proses ini dilakukan terus menerus hingga ditemukan posisi kosong (null) untuk meletakkan node baru. Dengan cara tersebut, struktur BST tetap teratur sesuai aturan bahwa subtree kiri memiliki nilai lebih kecil dan subtree kanan memiliki nilai lebih besar.

6. Jelaskan langkah-langkah pada method delete() saat menghapus sebuah node yang memiliki dua anak. Bagaimana method getSuccessor() membantu dalam proses ini?

Jawab: Ketika node yang akan dihapus memiliki dua child, proses penghapusan dilakukan dengan mencari node pengganti terlebih dahulu menggunakan method getSuccessor(). Successor adalah node dengan nilai terkecil pada subtree kanan dari node yang akan dihapus.

Langkah-langkahnya yaitu:

1. Cari node yang akan dihapus.
2. Cari successor menggunakan method getSuccessor().
3. Successor menggantikan posisi node yang dihapus.
4. Child kiri dari node yang dihapus dipindahkan menjadi child kiri successor.
5. Hubungan antar node diperbaiki agar struktur BST tetap valid.

Method getSuccessor() membantu mencari node pengganti yang paling tepat sehingga susunan data dalam Binary Search Tree tetap teratur setelah proses penghapusan dilakukan.

14.3.1 Percobaan 2

➤ Kode Program

> Kode Program class BinaryTreeArray06.java

```
package Pertemuan14;

public class BinaryTreeArray06 {
    Mahasiswa06[] dataMahasiswa;
    int idxLast;

    public BinaryTreeArray06() {
        this.dataMahasiswa = new Mahasiswa06[10];
    }

    public void populateData (Mahasiswa06 dataMhs[], int idxLast) {
        this.dataMahasiswa = dataMhs;
        this.idxLast = idxLast;
    }

    public void traverseInOrder (int idxStart) {
        if (idxStart <= idxLast) {
            if (dataMahasiswa[idxStart] != null) {
                traverseInOrder(2*idxStart+1);
                dataMahasiswa[idxStart].tampilInformasi();
                traverseInOrder(2*idxStart+2);
            }
        }
    }
}
```

> Kode Program class BinaryTreeArrayMain06.java

```
package Pertemuan14;

public class BinaryTreeArrayMain06 {
    public static void main(String[] args) {
        BinaryTreeArray06 bta = new BinaryTreeArray06();
        Mahasiswa06 mhs1 = new Mahasiswa06("244160121", "Ali", "A", 3.57);
        Mahasiswa06 mhs2 = new Mahasiswa06("244160185", "Candra", "C", 3.41);
        Mahasiswa06 mhs3 = new Mahasiswa06("244160221", "Badar", "B", 3.75);
        Mahasiswa06 mhs4 = new Mahasiswa06("244160220", "Dewi", "B", 3.35);

        Mahasiswa06 mhs5 = new Mahasiswa06("244160131", "Devi", "A", 3.48);
        Mahasiswa06 mhs6 = new Mahasiswa06("244160205", "Ehsan", "D", 3.61);
        Mahasiswa06 mhs7 = new Mahasiswa06("244160170", "Fizi", "B", 3.86);
    }
}
```

```

Mahasiswa06[] dataMahasiswas =
{mhs1,mhs2,mhs3,mhs4,mhs5,mhs6,mhs7,null,null,null};
int idxLast = 6;
bta.populateData(dataMahasiswas, idxLast);
System.out.println("\nInorder Traversal Mahasiswa: ");
bta.traverseInOrder(0);

```

➤ Hasil Running

```

Inorder Traversal Mahasiswa:
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.35
NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.41
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.48
NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.61
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.75
NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.86

```

Pertanyaan

1. Apakah kegunaan dari atribut data dan idxLast yang ada di class BinaryTreeArray?
Jawab: Atribut data digunakan untuk menyimpan node-node binary tree dalam bentuk array. Sedangkan idxLast digunakan untuk menyimpan indeks terakhir array yang sudah terisi data sehingga program mengetahui batas akhir data yang digunakan.

2. Apakah kegunaan dari method populateData()?
Jawab: Method populateData() digunakan untuk mengisi array binary tree dengan data awal yang sudah disediakan serta menentukan indeks terakhir data yang terisi (idxLast).

3. Apakah kegunaan dari method traverseInOrder()?
Jawab: Method traverseInOrder() digunakan untuk menampilkan isi binary tree dengan urutan traversal inorder yaitu:
 1. mengunjungi subtree kiri,
 2. mengunjungi root,
 3. mengunjungi subtree kanan.

Pada Binary Search Tree, traversal inorder menghasilkan data yang tampil secara terurut dari nilai terkecil ke terbesar.

4. Jika suatu node binary tree disimpan dalam array indeks 2, maka di indeks berapakah posisi left child dan right child masing-masing?
Jawab:
 Pada representasi binary tree menggunakan array berlaku rumus:
 - left child = $2 * i + 1$
 - right child = $2 * i + 2$

Jika node berada pada indeks 2:

- left child = $2(2) + 1 = 5$
- right child = $2(2) + 2 = 6$

Jadi left child berada di indeks 5 dan right child berada di indeks 6.

5. Apa kegunaan statement `int idxLast = 6` pada praktikum 2 percobaan nomor 4?

Jawab: Statement `int idxLast = 6` digunakan untuk menunjukkan bahwa indeks terakhir array yang terisi adalah indeks ke-6. Karena indeks array dimulai dari 0, maka jumlah data yang tersimpan di dalam array adalah 7 data.

6. Mengapa indeks $2*idxStart+1$ dan $2*idxStart+2$ digunakan dalam pemanggilan rekursif, dan apa kaitannya dengan struktur pohon biner yang disusun dalam array?

Jawab: Karena pada representasi binary tree menggunakan array terdapat aturan:

- child kiri suatu node berada pada indeks $2*i+1$
- child kanan suatu node berada pada indeks $2*i+2$

Rumus tersebut digunakan dalam pemanggilan rekursif agar program dapat mengakses posisi child kiri dan child kanan dari suatu node secara otomatis. Dengan cara ini, struktur binary tree tetap dapat direpresentasikan dengan benar di dalam array tanpa memerlukan pointer seperti pada linked list.

14.4 Tugas Praktikum

➤ Kode Program

> Modifikasi kode program class `BinaryTree06.java`

```
public void addRekursif(Mahasiswa06 mahasiswa) {  
    root = addRekursif(root, mahasiswa);  
}
```

```
public Node06 addRekursif(Node06 current, Mahasiswa06 mahasiswa) {  
    if (current == null) {  
        return new Node06(mahasiswa);  
    }  
    if (mahasiswa.ipk < current.mahasiswa.ipk) {  
        current.left = addRekursif(current.left, mahasiswa);  
    } else if (mahasiswa.ipk > current.mahasiswa.ipk) {  
        current.right = addRekursif(current.right, mahasiswa);  
    }  
    return current;  
}
```

```
public void cariMinIPK() {  
    if (root == null) {  
        System.out.println("Tree kosong");  
        return;  
    }  
}
```

```

    }
    Node06 current = root;
    while (current.left != null) {
        current = current.left;
    }
    current.mahasiswa.tampilInformasi();
}

public void cariMaxIPK() {
    if (root == null) {
        System.out.println("Tree kosong");
        return;
    }
    Node06 current = root;
    while (current.right != null) {
        current = current.right;
    }
    current.mahasiswa.tampilInformasi();
}

public void tampilMahasiswaIPKdiAtas(double ipkBatas) {
    tampilMahasiswaIPKdiAtas(root, ipkBatas);
}

public void tampilMahasiswaIPKdiAtas(Node06 node, double ipkBatas) {
    if (node != null) {
        tampilMahasiswaIPKdiAtas(node.left, ipkBatas);
        if (node.mahasiswa.ipk > ipkBatas) {
            node.mahasiswa.tampilInformasi();
        }
        tampilMahasiswaIPKdiAtas(node.right, ipkBatas);
    }
}
}

```

> Modifikasi kode program class BinaryTreeMain06.java

```

System.out.println("\nMahasiswa dengan IPK terkecil: ");
bst.cariMinIPK();
System.out.println("\nMahasiswa dengan IPK terbesar: ");
bst.cariMaxIPK();
System.out.println("\nMahasiswa dengan IPK di atas 3.50: ");
bst.tampilMahasiswaIPKdiAtas(3.50);

```

➤ Hasil Running

Daftar semua mahasiswa (in order traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Pencarian data mahasiswa:

Cari mahasiswa dengan ipk: 3.54 : Ditemukan

Cari mahasiswa dengan ipk: 3.22 : Tidak ditemukan

Daftar semua mahasiswa setelah penambahan 3 mahasiswa:

InOrder Traversal:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

PreOrder Traversal:

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

PostOrder Traversal:

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

Penghapusan data mahasiswa

Jika 2 anak, current =

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

Daftar semua mahasiswa setelah penghapusan 1 mahasiswa (in order traversal):

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.37

NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.46

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Mahasiswa dengan IPK terkecil:

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.21

Mahasiswa dengan IPK terbesar:

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

Mahasiswa dengan IPK di atas 3.50:

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.54

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.72

NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.85

➤ Kode Program

> Modifikasi kode program class BinaryTreeArray06.java

```
public void add(Mahasiswa06 data) {
    if (idxLast < dataMahasiswa.length - 1) {
        idxLast++;
        dataMahasiswa[idxLast] = data;
    } else {
        System.out.println("Array penuh");
    }
}

public void traversePreOrder(int idxStart) {
    if (idxStart <= idxLast) {
        if (dataMahasiswa[idxStart] != null) {
            dataMahasiswa[idxStart].tampilInformasi();
            traversePreOrder(2 * idxStart + 1);
            traversePreOrder(2 * idxStart + 2);
        }
    }
}
```

> Modifikasi kode program class BinaryTreeArray06.java

```
System.out.println("\nPreorder Traversal Mahasiswa: ");
bta.traversePreOrder(0);
```

➤ Hasil Running

Inorder Traversal Mahasiswa:

NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.35

NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.41

NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.48

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57

NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.61
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.75
NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.86

Preorder Traversal Mahasiswa:

NIM: 244160121 | Nama: Ali | Kelas: A | IPK: 3.57
NIM: 244160185 | Nama: Candra | Kelas: C | IPK: 3.41
NIM: 244160220 | Nama: Dewi | Kelas: B | IPK: 3.35
NIM: 244160131 | Nama: Devi | Kelas: A | IPK: 3.48
NIM: 244160221 | Nama: Badar | Kelas: B | IPK: 3.75
NIM: 244160205 | Nama: Ehsan | Kelas: D | IPK: 3.61
NIM: 244160170 | Nama: Fizi | Kelas: B | IPK: 3.86